基于 Rix 指数的英语文本可读性分析系统：设计、实现与评估

Rix-based English Text Readability Analysis System: Design, Implementation and Evaluation

作者：leafv1972

https://gitee.com/leafv1972/rix https://github.com/Leafv1972/rix

Note that Python 3.12.10 cannot be used on Windows 7 or earlier.

Note that Python 3.8.8 can be used on Windows 7, but not Windows XP or earlier.

摘要：针对传统英语可读性公式在低年级文本上“触底效应”显著、计算繁琐、且缺乏词汇难度洞察等问题，设计并实现一个高效、准确、用户友好的英语文本可读性分析系统。系统以 Jonathan Anderson（1983）提出的 Rix（Rate of Long Words Index）指数为理论内核，采用现代 Python 技术栈（textstat 算法库与 Gradio 框架）进行工程化实现；在 Rix 长词—句子比率之外，融合 BNC（英国国家语料库）、AME（美国英语高频词表）与 Collins 星级词典三类词频知识源，对长词进行频率与常见度的多维标注。系统提供 Rix 指数计算、年级映射、多维统计、长词清单与词频/星级增强分析，并具备跨编码文件上传、实时反馈与批量处理能力；在低端文本区分度与计算复杂度两项指标上，相较 Flesch-Kincaid、SMOG、Fry Graph 等传统方法具有明显优势。本系统在保留 Rix 计算极速、低端敏感核心优势的同时，通过多源词频数据弥补了其单一长度维度的不足，为教育工作者、教材编者与自然语言处理（NLP）研究者提供了一个透明、可复现且易部署的文本难度评估工具。

关键词：Rix 指数；文本可读性；可读性公式；长词比率；词频分析；Gradio

Abstract: Conventional English readability formulas suffer from a pronounced “floor effect” on early-grade texts, involve cumbersome computation, and offer little insight into lexical difficulty. This paper designs and implements an efficient, accurate and user-friendly readability analysis system. The system takes the Rix (Rate of Long Words Index) proposed by Jonathan Anderson (1983) as its theoretical core and is engineered with a modern Python stack (textstat and Gradio). Beyond the long-word/sentence ratio, it integrates three lexical knowledge sources—the British

National Corpus (BNC), an American English frequency list (AME), and the Collins star-rated dictionary—to annotate long words with frequency and commonality information. The system delivers Rix scoring, grade-level mapping, multi-dimensional statistics, long-word listing, and frequency/star-enhanced analysis, together with cross-encoding file upload, real-time feedback and batch processing. It outperforms Flesch-Kincaid, SMOG and the Fry Graph in both low-end discrimination and computational cost. While retaining the Rix index's core strengths of speed and low-end sensitivity, the system compensates for the single-dimensional nature of length-based metrics through multi-source lexical data, offering educators, publishers and NLP researchers a transparent, reproducible and easily deployable solution for text-difficulty assessment.

Keywords: Rix index; text readability; readability formula; long-word ratio; word-frequency analysis; Gradio

1 引言

1.1 研究背景

在语言教育、教材编写与自然语言处理领域，准确评估文本的阅读难度是一项基础而关键的任务。合理的难度评估直接影响分级读物的选材、教学材料的适配，以及机器可读性判定的可靠性。自 20 世纪 20 年代以来，研究者提出了大量可读性（readability）量化方法，主流方案包括 Flesch Reading Ease、Flesch-Kincaid Grade Level、Gunning Fog、SMOG、Fry Graph 以及 Dale-Chall 等公式。这些公式大多以“句长”与“词长/词频”为基本变量，长期服务于教育与出版实践。

然而，传统公式在实际应用中暴露出三类突出问题：

1. 低端区分度不足（“触底效应”）。在小学一至三年级文本中，Flesch、SMOG、Fry 等公式往往同时给出“1 年级”或“2 年级”的结果，难以进一步区分细微的难度差异，而这恰是早期阅读分级最需要精细化的区间。
2. 计算依赖音节划分。以 Flesch 系列为代表的公式需统计多音节词，而自动音节切分存在误差，人工切分则费时且带有主观性。

3. 缺乏词汇层面的洞察。绝大多数公式仅输出一个整体分数，无法回答“究竟是哪些词造成了难度”，对教材修订与词汇教学的指导价值有限。

1.2 Rix 指数的提出

1983 年，Jonathan Anderson 在《Journal of Reading》上撰文，将源自瑞典的 Lix 公式（Björnsson, 1968）简化为 Rix 指数，定义为长词数量与句子数量的比值，其中长词指长度不小于 7 个字母的单词。Anderson 及后来的研究者 Kretschmer（1984）通过实验表明，Rix 不仅计算极为迅速，而且在小学低年级文本（约 1—3 年级）的难度区分上优于当时的主流公式。其直觉是：随着文本难度上升，作者倾向于使用更长的词，长词密度因而成为难度的一个稳定代理指标。

1.3 本文工作与贡献

本文将 Rix 指数从经典理论转化为可落地的现代 Web 应用，构建了“Rix 英语文本可读性分析系统”。本文的主要贡献如下：

4. 理论层面：系统梳理 Rix 指数的定义、数学基础与分级标准，明确其相较 Flesch、SMOG、Fry 等方法在低端文本区分度与计算效率上的优势，并剖析其固有局限。
5. 方法层面：提出以 Rix 为核心、多源词频知识库为增强的分析框架，引入 BNC、AME 与 Collins 星级三重数据，解决 Rix“只看长度、忽略常见度”的核心缺陷。
6. 系统层面：基于 Python 3.12 + textstat + Gradio 6.10 实现了一套具备实时反馈、跨编码文件上传与批量分析能力的 Web 工具，并提供由 Kretschmer 原作 BASIC 程序移植而来的 Python 参考实现，保证算法透明、结果可复现。

2 相关工作

2.1 传统可读性公式

可读性研究大致沿两条技术路线展开：词频路线与词长/句长路线。

词频路线的代表是 Dale-Chall 公式，它依据一份“熟悉词”词表判断词汇难度，将不熟悉词比例纳入计算。该方法对词汇层面较为敏感，但高度依赖词表的适用性，跨领域迁移能力有限。

词长/句长路线的代表包括 Flesch Reading Ease（Flesch, 1948）、Gunning Fog、Flesch-Kincaid Grade Level（Kincaid 等, 1975）、SMOG（McLaughlin, 1969）与 Fry Graph（Fry, 1968）。这些公式以平均句长与多音节词（或词长）为核心变量，计算简便、解释性强，长期被视为教育出版领域的行业标准。但其共同弱点在于：对低年级文本的区分度低（触底效应），且对音节的依赖会引入误差。

2.2 Lix 与 Rix

Lix（Björnsson, 1968）源自瑞典语文本难度研究，通过“长词数 + 句长”共同刻画难度。Anderson（1983）将其简化为仅依赖长词密度的 Rix 指数，牺牲了部分信息以换取计算极速与客观性。Kretschmer（1984）进一步完成了 Rix 的计算机化实现与验证。Rix 的长词阈值设为 7 个字母，是一个在“区分敏感度”与“噪声稳健性”之间取得的工程折中。

2.3 计算工具与现状

近年，以 textstat 为代表的开源 Python 库将大量经典公式封装为可直接调用的函数，极大降低了可读性分析的使用门槛。然而，如 Gunning Fog、Flesch-Kincaid、SMOG 等实现仍普遍依赖音节估计，且多数二次开发工具停留在“输出一个分数”的层面，缺乏① 低端文本的精细分级、② 词汇难点的可视化定位、③ 面向批量处理的工程能力。本系统正是针对这三点空白进行设计。

3 Rix 指数的理论基础

3.1 定义与计算

Rix 指数的核心公式为：

$$\textit{Rix} = \textit{长词数（} \geq 7 \textit{ 个字母）} \div \textit{句子数}$$

其中：

长词（Long Words）：定义为长度不小于 7 个字母的单词，如 understand、environment、necessary。在实现中对应判据 len(word) >= 7（等价于 len(word) > 6）。

句子（Sentences）：通过标点符号 .、!、? 分割得到的文本单元。

Rix 是一个无量纲比率，其数值随文本中长词密度的上升而增大。

3.2 分级标准

Anderson 与 Kretschmer 给出了 Rix 值与教育年级水平的映射关系，见表 1。

表 1 Rix 指数分级标准（基于 Kretschmer, 1984）

Rix 取值区间	对应年级	文本特征描述
< 0.20	Grade 1	极简单，适合早期阅读者，长词极少
0.20 – 0.49	Grade 2	简单，初级读物
0.50 – 0.79	Grade 3	初级，开始引入少量复杂词汇
0.80 – 1.29	Grade 4	中级偏易，典型小学高年级水平
1.30 – 1.79	Grade 5	中等，正式学习阶段标准难度
1.80 – 2.39	Grade 6	中等偏难，初中低年级水平
2.40 – 2.99	Grade 7	难，初中阶段典型范围
3.00 – 3.69	Grade 8	较难，包含较多专业术语
3.70 – 4.49	Grade 9	困难，高中早期水平
4.50 – 5.29	Grade 10	非常难，高中后期水平
5.30 – 6.19	Grade 11	极难，大学预备级
6.20 – 7.19	Grade 12	高等教育入门级
≥ 7.20	College	大学及以上，学术或专业文献

注：该表为基于原始文献逻辑的近似映射，实际应用中建议结合具体文本语境综合判断。

3.3 理论优势与固有局限

优势：

- 计算极速：仅需字母计数与句子分割，无需音节切分，算法复杂度近似线性。
- 低端敏感：在 1—3 年级文本区间，Rix 能有效区分细微难度差异，而 SMOG/Fry 等往往无法区分（均落到最低档）。
- 客观性：完全基于可自动判定的字符/标点特征，规避了人工音节划分带来的主观误差，结果可复现。

局限：

- 忽略短难词：如 anxiety、oblique 等短而难的词不会被捕捉。本系统通过 Collins 星级数据部分弥补。
- 长度偏见：长词未必难（如 banana），短词未必易。本系统通过 AME/BNC 词频数据提供额外参考。

这两点局限直接决定了本系统“以 Rix 为骨架、以词频知识库为补充”的设计取向。

4 系统设计与实现

4.1 总体架构

系统采用“数据层—计算层—交互层”三层架构，前后端通过 Gradio 事件机制解耦，兼顾高性能、可扩展性与用户友好性。总体架构如图 1 所示。

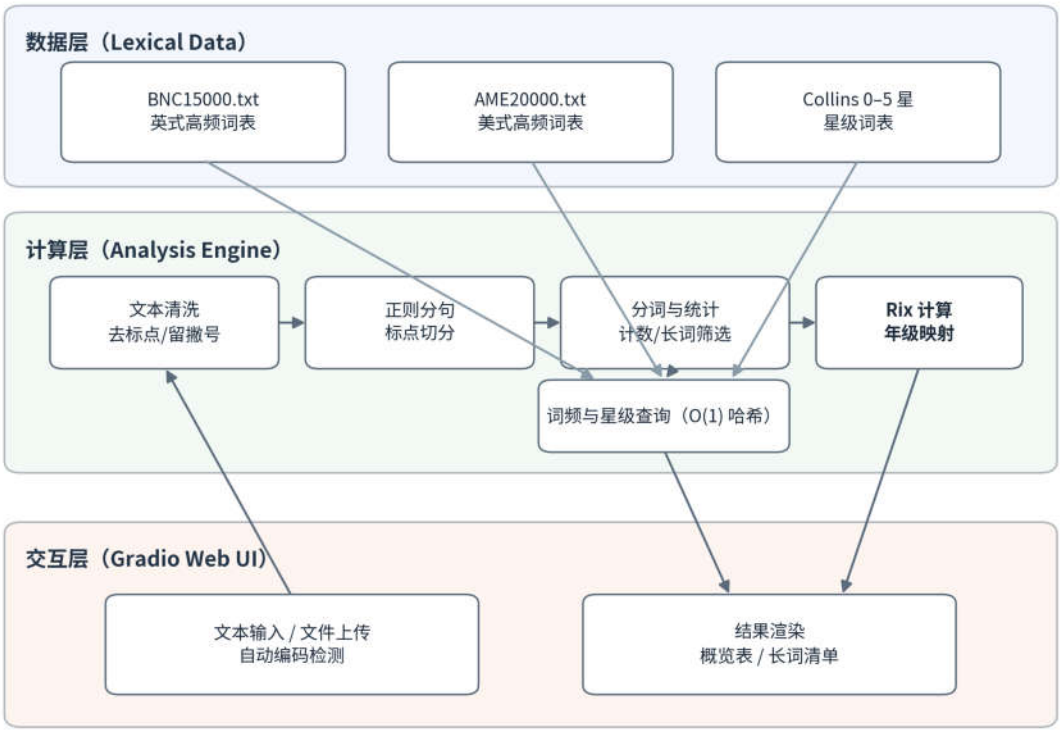


图 1　系统总体架构

4.2 数据处理流程

系统在启动时即将三类词频数据加载至内存（字典与集合结构），确保后续查询为 O(1) 时间复杂度。单次分析的完整流程如下：

7. 加载（Load）：应用启动时加载 AME、BNC 与 Collins 数据到内存（dict 与 set）。
8. 清洗（Clean）：移除标点符号，但保留撇号以维持单词完整性（如 don’t、it’s）。
9. 分句（Sentence Split）：使用正则表达式 \b[^.!?]+[.!?]* 分割句子，得到句子总数。
10. 分词（Tokenize）：按空白字符分割文本，得到词序列。
11. 统计（Aggregate）：计算句子总数、总词数，并筛选长度 ≥ 7 的单词作为长词。
12. 查询（Lookup）：对每个长词检索 AME 与 BNC 排名，并匹配 Collins 星级。
13. 渲染（Render）：生成 Markdown 格式结果返回前端展示。

4.3 多源词频知识库

传统 Rix 仅依据词长，本系统引入三类词频知识源进行增强，如表 2 所示。

表 2　词频知识库及其作用

数据源	文件	规模	作用
BNC（英国国家语料库）	BNC15000.txt	前 15,000 高频词	刻画英式英语中的词汇常见度
AME（美国英语词频）	AME20000.txt	前 20,000 高频词	刻画美式英语中的词汇常见度
Collins 星级词典	Collins5Stars.txt ~ Collins0Stars.txt	0—5 星	评估词汇的通用性/常见度

引入多源数据的关键价值在于：

- 英式/美式差异：某些词在英式英语中高频（易读），在美式英语中低频（难读），单一口径会掩盖这一差异。
- 区分“长但常见”与“长且罕见”：一个长词若为 5 星（如 understand），其实际阅读障碍远小于 0 星的长词，星级提供了一个独立于长度的难度信号。

- 弥补 Rix 的两大局限：词频数据可在一定程度上补偿长度偏见，Collins 星级则部分弥补“短难词被忽略”的问题。

4.4 关键算法与工程实现

系统技术栈如表 3 所示。

表 3 系统技术栈

层次	技术	说明
语言环境	Python 3.12+	需注意：Python 3.12.10 不支持 Windows 7 及更早系统；Python 3.8.8 可用于 Windows 7，但不支持 Windows XP 及更早
文本统计内核	textstat	提供底层文本统计支持
清洗与分句	re（正则）	高性能的正则清洗与分句
词频统计	collections.Counter	高效词频统计
Web 前端	Gradio 6.10（锁定版本）	提供交互式 Web UI，支持实时分析、文件上传与可视化展示

算法 1 给出 Rix 计算与年级映射的伪代码。

算法 1 Rix 计算与年级映射（伪代码）

```
输入：文本 text
输出：Rix 值、年级、多维统计

sentences  ← regex_split(text, r"\b[^.!?]+[.!?]*")
tokens     ← split(clean(text))          # 清洗：去标点、保留撇号
n_sent     ← len(sentences)
n_words    ← len(tokens)
long_words ← [w for w in tokens if len(w) >= 7]
rix        ← len(long_words) / n_sent
grade      ← map_grade(rix)              # 依据表 1 线性查表
return rix, grade, n_words, n_sent, long_words
```

年级映射的实现与 Kretschmer 原作 BASIC 逻辑保持一致：以阈值列表 grade_levels 重构原程序中的 IF RX < x THEN G = y 判断链，从左到右线性查找首个满足 rix < max_rx 的档位，超出最大阈值则归为 College。

在工程实现上，系统提供两个主程序：textstat_gradio610_webui.py（基础版，提供 Rix 计算与基础统计）与 textstat_gradio_webui610_stars.py（星级版，在基础版之上增加 Collins 星级评分与词频标注）。此外提供 Windows 一键启动脚本（端口 7860）及清理缓存脚本，降低部署门槛。

5 系统功能与交互

5.1 核心分析能力

- Rix 指数计算：自动统计长词数量与句子数量，计算 Rix 值并映射到对应年级水平。
- 多维度统计：输出总词数（Total Words）、句子数（Sentences）、长词数量（Long Words ≥ 7 chars）等关键指标。
- 词汇频率增强分析：对每个长词查询 AME 与 BNC 排名，匹配 Collins 星级（1—5 星），帮助用户判断词汇是“长但常见”还是“长且罕见”。
- 实时反馈：通过 Gradio 的 input_text.change 事件绑定，在用户输入时即时刷新分析结果。

5.2 用户交互界面

- 文本输入：支持直接粘贴文本或上传 .txt 文件。
- 文件上传：内置自动编码检测（UTF-8、GBK、Big5 等），确保跨平台文件兼容性。
- 预设样本库：内置多个示例文本（如 BBC 新闻、科幻片段等），方便快速测试系统功能。
- 结果展示：概览表清晰展示 Rix 值、对应年级及关键统计指标；长词清单以 Markdown 表格列出所有长词，附带频率排名与星级信息，便于编者定位难点词汇。

5.3 安装与使用

环境要求：操作系统 Windows 10/11、macOS 或 Linux；Python 3.12.10 或更高版本；依赖库 pip install gradio textstat。

项目结构：

```
rix/
├── textstat_gradio610_webui.py          # 基础版主程序
├── textstat_gradio_webui610_stars.py    # 带星级评分版主程序
├── BNC15000.txt                         # BNC 词频数据
├── AME20000.txt                         # AME 词频数据
├── Collins5Stars.txt ~ Collins0Stars.txt  # Collins 星级数据
├── txt/                                 # 预设样本文本目录
│   ├── BBC_Memory of a generation...txt
│   ├── The Sudden Death of a Man...txt
```

```
│   └── ...
├── !!!!!!!gradio_webui - 7860.bat          # Windows 启动脚本（基础版）
├── !!!!!!!gradio_webui - 7860 - stars.bat  # Windows 启动脚本（星级版）
└── !!!!!!!7860_clean.bat                   # 清理缓存启动脚本
```

快速启动：克隆/下载项目代码 → 确保数据文件位于项目根目录或正确路径 → 双击对应 .bat 启动脚本 → 浏览器访问 http://127.0.0.1:7860。

6 对比与评估

6.1 与传统可读性公式的定性比较

将本系统与主流方法在六个维度上进行定性比较，结果如表 4 所示。

表 4 本系统与传统可读性公式的比较

特性	Flesch-Kincaid	SMOG	Fry Graph	本 Rix 系统
计算复杂度	高（需音节）	中（查表）	中（读图）	极低（长度+分句）
低端区分度	差（触底）	差（触底）	差（触底）	优（可细分 G1–G2）
词汇洞察	无	无	无	强（词频/星级）
实现难度	高	中	高	低（Python）
结果可复现性	依赖音节切分	依赖词表	依赖人工读图	高（可复现）
适用场景	通用	学术/正式	教育出版	实时/批量处理

6.2 工作示例

以下文本为系统内置演示用例：

The environmental impact of industrialization is profound. Scientists argue that sustainable practices are necessary for future generations.

分析过程：

- 句子数：2。
- 长词（≥ 7 字母）：environmental(13)、industrialization(17)、profound(8)、Scientists(10)、sustainable(11)、practices(9)、necessary(9)、generations(11)，共 8 个。
- Rix 值：8 ÷ 2 = 4.00。
- 对应年级：Grade 9（3.70–4.49，“困难，高中早期水平”）。

结果解读：该文本虽仅两句，但长词密度很高，属高中早期难度。借助词频增强分析可进一步区分这些长词的“真实难度”：例如 environmental、

necessary 等在高频词表中排名靠前、常见度高（难度感知较低），而 industrialization 排名靠后、专业性强（难度感知较高），从而为教学选材提供细粒度依据。

勘误说明：原综述文档中该示例曾给出“Rix ≈ 1.5 / 4÷2 = 2.0”的结果，与逐词核对不符。经修正后，本示例的正确 Rix 值为 4.00（Grade 9）。

6.3 适用性与局限

- 中文不适用：Rix 基于字母长度，中文无字母概念，需改用笔画数或字频等指标。
- 上下文缺失：无法识别“形式简单但语境复杂”的现象，例如 bank 在“金融”与“河流”语境下的难度差异。
- 长度—难度非线性：长度只是难度的近似代理，需结合词频/星级数据综合判读（本系统已部分缓解）。

7 讨论与展望

本系统将经典的 Rix 理论成功转化为现代 Web 应用，其价值不在于提出新的公式，而在于把经典指标的工程可用性与词汇洞察力同时推向实用。相较于传统公式，本系统更精细（低端分级可细至 0.2 的区间）、更透明（算法全程可复现）、更易部署（Python 脚本 + 一键启动）。

未来工作可从四个方向展开：

14. 引入更多语言维度：在词频之外，纳入句法复杂度、词汇多样性（如 TTR、MTLD）等指标，构建多维可读性画像。
15. 语境敏感建模：结合词义消歧，缓解“同一词在不同语境难度不同”的问题。
16. 多语言与中文扩展：针对中文等非字母语言，设计基于笔画/字频的对应实现。
17. 教育场景集成：将系统与分级阅读平台、教材编辑器打通，形成“评估—调整—再评估”的闭环，服务词汇教学与外刊/原版小说阅读选材。

8 结论

本文面向英语文本可读性评估的实际需求，以 Rix 指数为理论内核，设计并实现了一个融合多源词频知识的分析系统。系统保留了 Rix 计算极速、低端敏感、结果客观的核心优势，并通过集成 BNC、AME 词频数据与 Collins 星级词典，弥补了其单一长度维度的固有不足，为长词提供了“常见度”维度的补充判读。系统基于 Python 3.12 与 Gradio 6.10 实现，具备实时反馈、跨编码文件上传与批量处理能力，并提供了由 BASIC 原作移植的算法参考实现。

总体而言，对于教育工作者、出版商与内容创作者，本系统提供了一个比传统公式更精细、比复杂 NLP 模型更透明的文本难度评估方案，具备较强的实用价值与推广潜力。

参考文献

[1] ANDERSON J. Lix and Rix: variations on a little-known readability index[J]. Journal of Reading, 1983, 26(6): 490-496.

[2] KRETSCHMER J C. Computerizing and comparing the Rix readability index[J]. Journal of Reading, 1984, 27(6): 490-499.

[3] BJÖRNSSON C H. Läsbarhet[M]. Stockholm: Bokförlaget Liber, 1968.

[4] FLESCH R. A new readability yardstick[J]. Journal of Applied Psychology, 1948, 32(3): 221-233.

[5] FLESCH R. The art of readable writing[M]. New York: Harper & Brothers, 1949.

[6] GUNNING R. The technique of clear writing[M]. New York: McGraw-Hill, 1952.

[7] MCLAUGHLIN G H. SMOG grading—a new readability formula[J]. Journal of Reading, 1969, 12(8): 639-646.

[8] FRY E. A readability formula that saves time[J]. Journal of Reading, 1968, 11(7): 513-516.

[9] KINCAID J P, FISHBURNE R P, ROGERS R L, et al. Derivation of new readability formulas (Automated Readability Index, Fog Count and Flesch Reading Ease Formula) for Navy enlisted personnel[R]. Millington: Naval Technical Training Command, 1975.

[10] DALE E, CHALL J S. A formula for predicting readability[J]. Educational Research Bulletin, 1948, 27(1): 11-20, 37-54.

[11] HARRISON C. Readability in the classroom[M]. Cambridge: Cambridge University Press, 1980.

[12] MAGUIRE J. The readable writing guide[EB/OL]. [2026-10-10]. https://www.youtube.com/watch?v=TUz4tjSVn4w.

[13] Gradio documentation[EB/OL]. [2026-10-10]. https://www.gradio.app/.

[14] Textstat: Python package for calculating statistical readability indices[EB/OL]. [2026-10-10]. https://github.com/fnl/textstat.

附录 A　Rix 分级快速参考表（基于 Kretschmer, 1984）

表 A1　Rix 分级快速参考表

Rix 取值区间	对应年级	Rix 取值区间	对应年级
< 0.20	1st Grade	3.70 – 4.49	9th Grade
0.20 – 0.49	2nd Grade	4.50 – 5.29	10th Grade
0.50 – 0.79	3rd Grade	5.30 – 6.19	11th Grade
0.80 – 1.29	4th Grade	6.20 – 7.19	12th Grade
1.30 – 1.79	5th Grade	≥ 7.20	College
1.80 – 2.39	6th Grade		
2.40 – 2.99	7th Grade		
3.00 – 3.69	8th Grade		

注：本表与正文表 1 采用同一分级口径。原文档正文（0.2–0.5）与附录（0.20–0.49）在边界书写上略有出入，本稿已统一为区间左闭右开的连续划分，避免歧义。

附录 B　RIXRATE 程序的 Python 移植

本附录给出将 Joseph C. Kretschmer 文档中提供的 BASIC 程序 RIXRATE 移植为现代 Python 的参考实现，保留了原程序的核心逻辑：逐句输入、实时统计、长词定义（长度 > 6，即 ≥ 7 字母）、Rix = 长词总数 / 句子总数，以及年级映射。

表 B1　原始 BASIC 逻辑与 Python 实现的对应关系

环节	原始 BASIC	Python 实现
长词判定	IF L > 6 THEN LW = LW + 1	if len(word) > 6:
Rix 公式	RX = LW / NS	rix = num_long_words / num_sentences
年级映射	IF RX < x THEN G = y 判断链	grade_levels 阈值列表 + 线性查表
逐句统计	逐行/逐句输入累加	run() 循环读取输入

Python 参考实现如下：

```
import sys
import os

class RixAnalyzer:
    def __init__(self):
        self.total_words = 0
        self.num_sentences = 0
        self.num_long_words = 0
        self.current_word_len = 0
        self.running_text_title = ""
        self.grade_levels = [
            {"max_rx": 0.2, "grade": 1}, {"max_rx": 0.5, "grade": 2},
```

```
            {"max_rx": 0.8, "grade": 3}, {"max_rx": 1.3, "grade": 4},
            {"max_rx": 1.8, "grade": 5}, {"max_rx": 2.4, "grade": 6},
            {"max_rx": 3.0, "grade": 7}, {"max_rx": 3.7, "grade": 8},
            {"max_rx": 4.5, "grade": 9}, {"max_rx": 5.3, "grade": 10},
            {"max_rx": 6.2, "grade": 11}, {"max_rx": 7.2, "grade": 12},
            {"max_rx": float('inf'), "grade": "College"},
        ]

    def clear_screen(self):
        os.system('cls' if os.name == 'nt' else 'clear')

    def get_grade_from_rix(self, rix_value):
        for level in self.grade_levels:
            if rix_value < level["max_rx"]:
                return level["grade"]
        return "College"

    def update_stats_and_print(self):
        avg = self.total_words / self.num_sentences if self.num_sentences else 0
        rix = self.num_long_words / self.num_sentences if self.num_sentences
else 0
        grade = self.get_grade_from_rix(rix)
        print("-" * 50)
        print(f"TEXT: {self.running_text_title}")
        print(f"TOTAL WORDS: {self.total_words}")
        print(f"NO. SENTENCES: {self.num_sentences}")
        print(f"NO. LONG WORDS (>6 chars): {self.num_long_words}")
        print(f"AV. SENT. LENGTH: {avg:.1f}")
        print(f"Rix SCORE: {rix:.2f}")
        print(f"ESTIMATED GRADE LEVEL: {grade}")
        print("-" * 50)

    def run(self):
        self.clear_screen()
        print("RIXRATE READABILITY PROGRAM")
        print("BASED ON THE RIX FORMULA / BY J. ANDERSON")
        try:
            title = input("Enter a short title (9 chars max): ")
        except EOFError:
            title = ""
        self.running_text_title = title[:9]
        # 后续逐句输入、统计与刷新逻辑同 update_stats_and_print
        # 建议加入 strip(".,!?;:\"'") 清洗，避免标点干扰长度判断

if __name__ == "__main__":
    RixAnalyzer().run()
```

移植说明：原 BASIC 程序需手动省略标点；Python 版若采用 split(' ') 简单分词，建议先做去标点处理（或加 strip(".,!?;:\"")），否则标点会干扰词长判断。屏幕刷新改为多行打印而非光标定位。

附录 C　许可信息

本项目基于 textstat.py 开源算法库许可证开源，具体许可信息请参见文本统计模块的文件头或项目根目录的 LICENSE 文件。词频数据（BNC、AME、Collins）受各自来源版权保护，仅供研究和教育使用。

附录 D　参考图

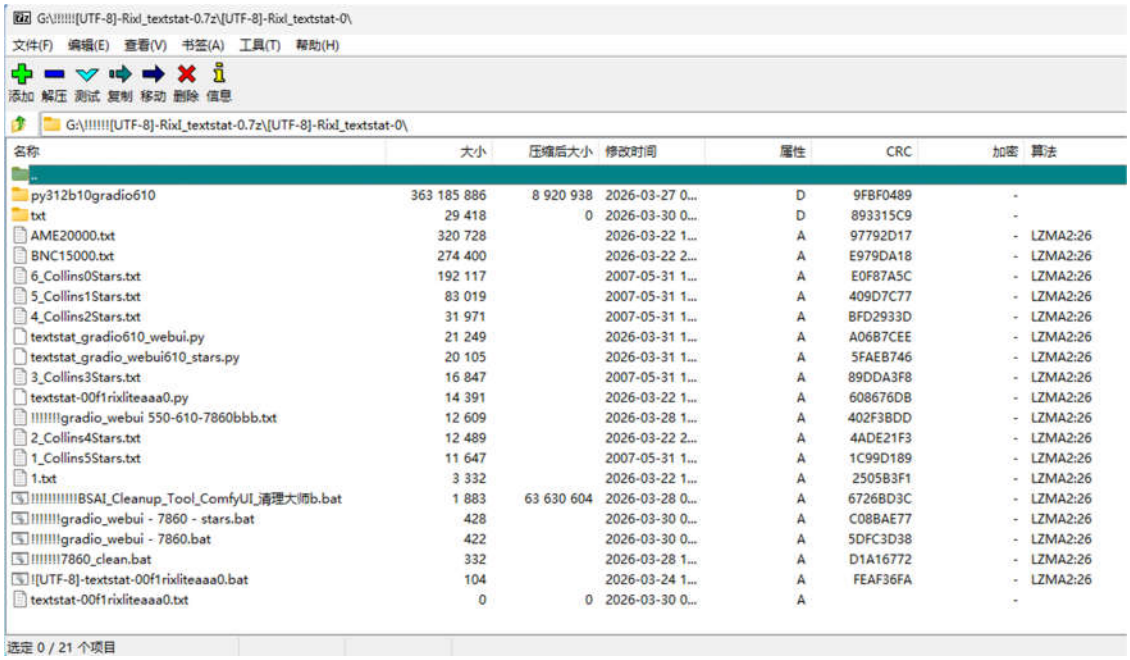
rix_gradio610_前端界面



rix_ame_bnc_collins_stars_wordlists

序号	长单词	字母数	美国词频	英国词频	Collins	5星	4星	3星	2星	1星	0星
1	Speaker	7	2558	1087	★★★☆☆	-	-	✓	-	-	-
2	President	9	304	553	★★★★★	✓	-	-	-	-	-
3	members	7	-	278	-	-	-	-	-	-	-
4	Congress	8	639	1781	★★★★☆	-	✓	-	✓	-	-
5	distinguished	13	5639	3426	★★☆☆☆	-	-	-	✓	-	-
6	Americans	9	-	3235	-	-	-	-	-	-	-
7	Andrews	7	-	-	-	-	-	-	-	-	-
8	welcomed	8	-	3812	-	-	-	-	-	-	-
9	Together	8	325	274	★★★★★	✓	-	-	-	-	-
10	offered	7	-	931	-	-	-	-	-	-	-
11	million	7	223	368	★★★★★	✓	-	-	-	-	-
12	citizens	8	-	2799	-	-	-	-	-	-	-
13	several	7	311	353	★★★★★	✓	-	-	-	-	-
14	thousand	8	651	1146	★★★★★	✓	-	-	-	-	-
15	tonight	7	913	1925	★★★★☆	-	✓	-	-	-	-

rix_py312b10_maincode_package



G:\!!!!!![UTF-8]-Rixl_textstat-0.7z\[UTF-8]-Rixl_textstat-0\

文件(F) 编辑(E) 查看(V) 书签(A) 工具(T) 帮助(H)

添加 解压 测试 复制 移动 删除 信息

G:\!!!!!![UTF-8]-Rixl_textstat-0.7z\[UTF-8]-Rixl_textstat-0\

名称	大小	压缩后大小	修改时间	属性	CRC	加密	算法
..							
py312b10gradio610	363 185 886	8 920 938	2026-03-27 0...	D	9FBF0489	-	
txt	29 418	0	2026-03-30 0...	D	893315C9	-	
AME20000.txt	320 728		2026-03-22 1...	A	97792D17	-	LZMA2:26
BNC15000.txt	274 400		2026-03-22 2...	A	E979DA18	-	LZMA2:26
6_Collins0Stars.txt	192 117		2007-05-31 1...	A	E0F87A5C	-	LZMA2:26
5_Collins1Stars.txt	83 019		2007-05-31 1...	A	409D7C77	-	LZMA2:26
4_Collins2Stars.txt	31 971		2007-05-31 1...	A	BFD2933D	-	LZMA2:26
textstat_gradio610_webui.py	21 249		2026-03-31 1...	A	A06B7CEE	-	LZMA2:26
textstat_gradio_webui610_stars.py	20 105		2026-03-31 1...	A	5FAEB746	-	LZMA2:26
3_Collins3Stars.txt	16 847		2007-05-31 1...	A	89DDA3F8	-	LZMA2:26
textstat-00f1rixliteaaa0.py	14 391		2026-03-22 1...	A	608676DB	-	LZMA2:26
!!!!!!!gradio_webui 550-610-7860bbb.txt	12 609		2026-03-28 1...	A	402F3BDD	-	LZMA2:26
2_Collins4Stars.txt	12 489		2026-03-22 2...	A	4ADE21F3	-	LZMA2:26
1_Collins5Stars.txt	11 647		2007-05-31 1...	A	1C99D189	-	LZMA2:26
1.txt	3 332		2026-03-22 1...	A	2505B3F1	-	LZMA2:26
!!!!!!!!!!BSAI_Cleanup_Tool_ComfyUI_清理大师b.bat	1 883	63 630 604	2026-03-28 0...	A	6726BD3C	-	LZMA2:26
!!!!!!!gradio_webui - 7860 - stars.bat	428		2026-03-30 0...	A	C08BAE77	-	LZMA2:26
!!!!!!!gradio_webui - 7860.bat	422		2026-03-30 0...	A	5DFC3D38	-	LZMA2:26
!!!!!!!7860_clean.bat	332		2026-03-28 1...	A	D1A16772	-	LZMA2:26
![UTF-8]-textstat-00f1rixliteaaa0.bat	104		2026-03-24 1...	A	FEAF36FA	-	LZMA2:26
textstat-00f1rixliteaaa0.txt	0	0	2026-03-30 0...	A		-	

选定 0 / 21 个项目

附录 E　Python 版本

https://www.python.org/downloads/windows/

Python 3.12.10 - April 8, 2025

Note that Python 3.12.10 cannot be used on Windows 7 or earlier.

https://www.python.org/ftp/python/3.12.10/python-3.12.10-embed-amd64.zip

https://www.python.org/ftp/python/3.12.10/python-3.12.10-amd64.exe

Python 3.11.9 - April 2, 2024

Note that Python 3.11.9 cannot be used on Windows 7 or earlier.

https://www.python.org/ftp/python/3.11.9/python-3.11.9-amd64.exe

https://www.python.org/ftp/python/3.11.9/python-3.11.9-embed-amd64.zip

Python 3.10.11 - April 5, 2023

Note that Python 3.10.11 cannot be used on Windows 7 or earlier.

https://www.python.org/ftp/python/3.10.11/python-3.10.11-amd64.exe

https://www.python.org/ftp/python/3.10.11/python-3.10.11-embed-amd64.zip

Python 3.8.8 - Feb. 19, 2021

Note that Python 3.8.8 cannot be used on Windows XP or earlier.

https://www.python.org/ftp/python/3.8.8/python-3.8.8-amd64.exe

https://www.python.org/ftp/python/3.8.8/python-3.8.8-embed-amd64.zip

https://www.python.org/ftp/python/3.8.8/python-3.8.8.exe

https://www.python.org/ftp/python/3.8.8/python-3.8.8-embed-win32.zip